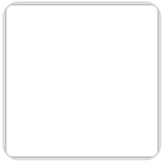
You said:



https://github.com/vishalh06/22_sliding_puzzle
This is the repo i have forked from the teacher.
I have attached the png of the deliverables etc. i want you to explain in 2 sentences what this exactly is about.
And then draft a work plan from start to finish.
We will not start yet, only when i say let's start, you help me out with each of the steps guiding me throughout until i finish it.

Worked for 45s >

ChatGPT said:

This is an individual lab where you use an AI coding tool to repair a Python sliding-puzzle game and add every feature required by its README, aiming to get each task working within three to four prompting attempts. You must demonstrate the game before and after your changes, make separate commits for each task, and submit the updated code and chat history.

I could read your attached instructions, but GitHub access failed, so **the exact bugs and required features still need to be checked in the README**.

Our work plan, from start to finish:

Step	What we'll do	Completion check
1. Confirm requirements	Read the README and make a checklist of every repair, feature, and submission requirement.	Nothing required is missing from the checklist.
2. Prepare your computer	Check Python, Git, and your editor; clone your existing fork and install its dependencies.	The project runs locally, and Git points to your personal fork.
3. Run the original game	Understand the controls and observe its broken behaviour without editing the code.	We know what needs fixing.
4. Record the "before" video	Capture **10 seconds of the original game**, showing the problem where possible.	The video is saved **before any code changes**.

Step	What we'll do	Completion check
5. Complete each README task	Write a focused AI prompt, apply the suggested changes, review them, and test the result. Add required features **one at a time**, keeping the prompts and responses.	Each task meets its requirement; aim for three to four attempts.
6. Commit each completed task	Make a descriptive, separate Git commit immediately after verifying each task.	Each task has its own commit.
7. Check the complete game	Test all required features together and confirm earlier fixes still work.	The full README checklist passes.
8. Record the "after" video	Capture **10 seconds of the finished game**, demonstrating the fixes and added features.	The final video is saved and plays correctly.
9. Prepare the submission	Export the complete AI chat history as a **DOC/DOCX or PDF** and gather both videos and the updated code. Check whether the README additionally requires a chat link.	All submission files are ready.
10. Push and submit	Push the task commits to your personal fork and upload the deliverables to your **Lab repository's `Lab-4` folder**.	Verify the uploaded files and commit history on GitHub. **Do not raise a PR to `SETAPESU26`.**

The listed lab duration is **60 minutes**. Your game fork and your Lab submission repository may be separate repositories; we'll confirm that before uploading.

When you say **"let's start,"** I'll guide you through one step at a time, including the exact commands and when to record, commit, and save each deliverable.

You said:

okay lets start

ChatGPT said:

Let's start with **Step 1: checking the exact requirements**.

Open your repository, scroll down to the **README**, and paste its entire contents here so I can identify every required fix and feature.

Also tell me whether you're doing this on **Windows or Ubuntu**.

Don't edit the code yet—we must record the 10-second "before" video first.

You said:

Scenario 22 — Sliding Puzzle

A terminal sliding puzzle with configurable board size and legal blank-space moves.

Provided files

- `main.py` — entry point.
- `game.py` — interaction, timer, and move tracking.
- `puzzle.py` — puzzle representation and movement rules.
- `requirements.txt` — dependency declaration.

Setup

```
python main.py
```

Before changing the code

Start several puzzles and inspect the initial boards. Think about whether arbitrary
permutations can always be reached from the solved state using legal moves.

Task 1 — Guarantee solvable starting states

Change puzzle initialisation so every generated board is reachable from the solved
arrangement. Prefer generating the solved board and applying legal blank moves to
scramble it.

Done when: repeated fresh puzzles are solvable in principle and the puzzle never
starts in an unreachable parity state.

Task 2 — Complete puzzle lifecycle

Detect the solved arrangement and end the game cleanly. Prevent commands from changing
state after completion.

Task 3 — Size, timer, and move modes

Add 3x3, 4x4, and 5x5 modes. Track moves and elapsed time consistently across all sizes.
Do not reset session values accidentally when only the board is recreated.

Task 4 — Valid-action feedback

Only report a successful slide when a tile actually moved into the blank. Invalid moves
must not increment the move count or generate success feedback.

Required testing

Generate many fresh boards, test all sizes, solve a small board, attempt impossible
moves, verify move counts, verify timer behaviour, test invalid commands, and quit.

LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while
retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.
- Do not replace the whole project with an unrelated implementation.
- Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Submission checklist

- [ ] Task 1 completed and the original defect was reproduced and fixed.
- [ ] Tasks 2–4 completed and tested.
- [ ] Boundary and invalid-input cases tested.
- [ ] No unnecessary external dependencies added.
- [ ] No persistent storage added.

- [ ] Code remains understandable and modular.
- [ ] Complete LLM chat-history link included.

Folder structure

```
scenario-10-sliding-puzzle/
├── README.md
├── requirements.txt
├── main.py
├── game.py
└── puzzle.py
```

Submission Checklist

Submission is only the following three things:

- [ ] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [ ] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [ ] The Chat/LLM used page link, with the complete chat history

ChatGPT said:

The README confirms **four tasks**: generate solvable boards, detect completion and stop further moves, support 3×3/4×4/5×5 with correct timing and move counts, and give accurate feedback for valid and invalid moves.

We'll keep the existing project structure, use only in-memory state, and make **one separate commit per task**. For submission, we'll include both 10-second videos, updated code, the exported chat history required by your handout, **and the complete chat link required by the README**.

Next: tell me these two things:

1. Are you using **Windows or Ubuntu**?
2. Have you already cloned/downloaded the repository onto your computer?

Then I'll give you the exact setup commands. **Don't modify any code yet—we'll inspect it, run the original version, and record the "before" video first.**

You said:

windows and should i downliad requirements in venv? i have cloned it already.

ChatGPT said:

Yes—**install the packages listed in `requirements.txt` inside a virtual environment**.

Open your cloned project folder in VS Code, then select **Terminal → New Terminal** and run these commands one at a time:

```
py -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

If activation gives a "running scripts is disabled" error, run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Then repeat the activation command. Once you see `(.venv)` in the terminal, run:

```
python -m pip install -r requirements.txt
```

Don't change the code yet. After installation, paste the contents of `main.py`, `game.py`, and `puzzle.py` here, each in a separate code block. We'll inspect the original code before running it and recording the required "before" video.

You said:

done next

ChatGPT said:

Next, **send me the original code so we can inspect it before making changes**.

In VS Code, open these files and paste each file's complete contents here, labelled with its filename:

- `main.py`
- `game.py`
- `puzzle.py`

You can also attach the three files directly. After reviewing them, I'll guide you through running the game and recording the **10-second "before" video**.

You said:

game.py

```
import time
from puzzle import Puzzle


class SlidingPuzzle:
    def __init__(self):
        self.size = 4
```

```
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print("Moves:", self.moves, " Time:", int(time.monotonic() -
self.started), "s")

    def run(self):
        print("Sliding Puzzle — W/A/S/D moves the tile into the blank.
Q quits.")
        while True:
            self.display()
            if self.puzzle.solved():
                print("Solved!")
                return
            key = input("> ").strip().lower()
            if key == "q":
                return
            if key not in "wasd":
                print("Use W/A/S/D.")
                continue
            if self.puzzle.move(key):
                self.moves += 1
            else:
                print("That move is not possible.")
```

main.py

```
from game import SlidingPuzzle

if __name__ == "__main__":
    SlidingPuzzle().run()
```

puzzle.py

```
import random


class Puzzle:
    def __init__(self, size=4):
```

```
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        tiles = list(range(1, self.size * self.size)) + [0]
        random.shuffle(tiles)  # intentional solvability defect
        return [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def move(self, direction):
        r, c = self.blank_pos()
        dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}[direction]
        nr, nc = r + dr, c + dc
        if not (0 <= nr < self.size and 0 <= nc < self.size):
            return False
        self.board[r][c], self.board[nr][nc] = self.board[nr][nc],
self.board[r][c]
        return True

    def solved(self):
        return sum(self.board, []) == list(range(1, self.size * self.size)) +
[0]
```

when do i have to screen record.

ChatGPT said:

Record now, before changing any code. First run the original game in your activated terminal:

```
python main.py
```

For the **10-second "before" video**:

1. Press **Windows + Shift + R** to open Snipping Tool's screen recorder, select the terminal area, and start recording.
2. Show the initial board, enter a few `w`, `a`, `s`, or `d` commands, then enter an invalid command such as `x`.
3. Stop after 10 seconds and save it as **`before.mp4`** outside the project folder for now.

The main defect is that `random.shuffle()` can create an unsolvable board. Simply showing a shuffled board doesn't prove that particular board is unsolvable; after saving this recording, we'll check the original generator's solvability without modifying its code, so we also satisfy the requirement to reproduce the defect.

Tell me once the video is saved.

You said:

done. now we'll start working on the features, one by one commiting them simultaneously.

ChatGPT said:

Yes—we'll complete and test **one task at a time, then commit it before moving on**.

Task 1: guarantee solvable starting boards. The current `random.shuffle()` can produce unreachable arrangements. We'll replace it with legal moves from the solved board, so every generated puzzle is solvable.

First, **reproduce the original defect**. Paste this entire block into your activated PowerShell terminal:

```
@'
from puzzle import Puzzle

def solvable(board):
    n = len(board)
    tiles = [x for row in board for x in row if x != 0]
    inversions = sum(
        tiles[i] > tiles[j]
        for i in range(len(tiles))
        for j in range(i + 1, len(tiles))
    )
    blank_row_from_bottom = n - next(
        r for r, row in enumerate(board) if 0 in row
    )
    if n % 2:
        return inversions % 2 == 0
    return (inversions + blank_row_from_bottom) % 2 == 1

for size in (3, 4, 5):
    failures = sum(not solvable(Puzzle(size).board) for _ in range(100))
    print(f"{size}x{size}: {failures}/100 boards are UNSOLVABLE")
'@ | python -
```

Save a screenshot of the output as evidence of the original defect. You should see some unsolvable boards.

Then, in **`puzzle.py`**, replace **only the `make_board()` method** with this:

```
    def make_board(self):
        tiles = list(range(1, self.size * self.size)) + [0]
        self.board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        opposite = {"w": "s", "s": "w", "a": "d", "d": "a"}
        previous = None

        for _ in range(self.size * self.size * 40):
            r, c = self.blank_pos()
            legal = []

            if r > 0:
                legal.append("w")
            if r < self.size - 1:
                legal.append("s")
            if c > 0:
                legal.append("a")
            if c < self.size - 1:
                legal.append("d")

            if previous is not None:
                legal = [
                    direction for direction in legal
                    if direction != opposite[previous]
                ]

            direction = random.choice(legal)
            self.move(direction)
            previous = direction

        if self.solved():
            self.move("w")

        return self.board
```

This starts with the solved arrangement and scrambles it using valid moves. It avoids immediately reversing the previous move and ensures the starting board isn't already solved; these scramble moves do **not** affect the player's move count.

Verify and commit:

1. Save the file and rerun the same testing block. All three sizes must report `0/100 boards are UNSOLVABLE`.
2. Run the game and check that movement still works:

   ```
   python main.py
   ```

3. Quit with `q`, then run:

   ```
   git diff -- puzzle.py
   ```

   ```
   git add puzzle.py
   git commit -m "Task 1: Guarantee solvable starting boards"
   ```

Send me the **test output and commit confirmation**, then we'll proceed to Task 2.

You said:

1 selection

okay i stopped right here and havent done anyhting after which u told me to like task1- guarantee solvable started boards etcetc yet. i closed that terminal. i did take the screen recording for before video. lets start from task 1 and tell me what to setup again, since i closed the terminal many days back.

ChatGPT said:

Your "**before" recording is already saved**, so we'll continue from there. Closing the terminal doesn't delete your virtual environment or installed packages.

1. Open **VS Code → File → Open Folder** and select your cloned `22_sliding_puzzle` folder.
2. Open **Terminal → New Terminal**. Make sure the terminal is PowerShell and its path ends in your project folder.
3. Activate the existing virtual environment:

   ```
   .\.venv\Scripts\Activate.ps1
   ```

   If you get a "running scripts is disabled" error, run these commands:

   ```
   Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
   .\.venv\Scripts\Activate.ps1
   ```

   You should now see `(.venv)` at the beginning of the terminal prompt. **You don't need to recreate the environment or reinstall requirements.**

4. Check your repository's current state:

```
git status
git log -5 --oneline
git remote -v
```

5. Display the current puzzle code:

```
Get-Content .\puzzle.py
```

Paste the output from steps 4 and 5 here. We'll then reproduce the original solvability defect, apply only the Task 1 fix, test it, and make its separate commit.

You said:

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
.\.venv\Scripts\Activate.ps1
.\.venv\Scripts\Activate.ps1 : The term '.\.venv\Scripts\Activate.ps1' is
not recognized as the name of a cmdlet, function, script file,
or operable program. Check the spelling of the name, or if a path
was included, verify that the path is correct and try again.
At line:1 char:1
+ .\.venv\Scripts\Activate.ps1
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound:
(.\.venv\Scripts\Activate.ps1:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Your prompt already shows `(venv)`, so your environment appears active. The error occurred because the command used `.venv`, while yours is likely named `venv`.

Skip activation and run:

```
git status
git log -5 --oneline
git remote -v
Get-Content .\puzzle.py
```

Paste the output here so we can begin Task 1.

You said:

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git status
>> git log -5 --oneline
>> git remote -v
>> Get-Content .\puzzle.py
```

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/
        before.mp4
        venv/

nothing added to commit but untracked files present (use "git add"
to track)
75858c1 (HEAD -> main, origin/main, origin/HEAD) Updated
scenario title
56c88a9 Initial commit
origin  https://github.com/vishalh06/22_sliding_puzzle.git (fetch)
origin  https://github.com/vishalh06/22_sliding_puzzle.git (push)
import random


class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        tiles = list(range(1, self.size * self.size)) + [0]
        random.shuffle(tiles)  # intentional solvability defect
        return [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def move(self, direction):
        r, c = self.blank_pos()
        dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}[direction]
        nr, nc = r + dr, c + dc
        if not (0 <= nr < self.size and 0 <= nc < self.size):
            return False
        self.board[r][c], self.board[nr][nc] = self.board[nr][nc],
self.board[r][c]
        return True
```

```
    def solved(self):
        return sum(self.board, []) == list(range(1, self.size * self.size)) +
[0]
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Your code is still unchanged, and `origin` correctly points to **your fork**. We'll first reproduce the Task 1 defect before fixing it.

Paste this entire block into the same PowerShell terminal. It checks 100 original boards for each required size without editing any files:

```
@'
from puzzle import Puzzle

def is_solvable(board):
    size = len(board)
    tiles = [tile for row in board for tile in row if tile != 0]
    inversions = sum(
        tiles[i] > tiles[j]
        for i in range(len(tiles))
        for j in range(i + 1, len(tiles))
    )

    if size % 2 == 1:
        return inversions % 2 == 0

    blank_row = next(r for r, row in enumerate(board) if 0 in row)
    row_from_bottom = size - blank_row
    return (inversions + row_from_bottom) % 2 == 1

for size in (3, 4, 5):
    unsolvable = sum(
        not is_solvable(Puzzle(size).board)
        for _ in range(100)
    )
    print(f"{size}x{size}: {unsolvable}/100 boards are unsolvable")
'@ | python -
```

Some boards should be reported as **unsolvable**, confirming that random shuffling creates arrangements that legal moves cannot solve.

Send me that output next. Keep `before.mp4` where it is; when committing, we'll stage specific files so `venv/` and `__pycache__/` aren't included.

You said:

before.mp4
File

okay lets not start yet, tell me if this video is correct and if it is just say yes. and ill start task1 with the commands u already gave me.

Worked for 14s

ChatGPT said:

The gameplay recording is suitable, but it's **16.7 seconds** long—trim it to **10 seconds** to match the submission requirement.

You said:

3x3: 61/100 boards are unsolvable
4x4: 58/100 boards are unsolvable
5x5: 49/100 boards are unsolvable
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>

ChatGPT said:

That confirms the original defect: **random shuffling produces unsolvable boards in all three sizes.**

Now complete **Task 1**.

1. In `puzzle.py`, replace **only** `make_board()` with the following. Keep the indentation exactly as shown:

```
def make_board(self):
    tiles = list(range(1, self.size * self.size)) + [0]
    board = [
        tiles[r * self.size:(r + 1) * self.size]
        for r in range(self.size)
    ]

    blank_r = self.size - 1
    blank_c = self.size - 1
    previous_blank = None

    for _ in range(self.size * self.size * 40):
        neighbours = [
            (r, c)
            for r, c in (
                (blank_r - 1, blank_c),
```

```
                (blank_r + 1, blank_c),
                (blank_r, blank_c - 1),
                (blank_r, blank_c + 1),
            )
            if 0 <= r < self.size and 0 <= c < self.size
            and (r, c) != previous_blank
        ]

        next_r, next_c = random.choice(neighbours)
        board[blank_r][blank_c], board[next_r][next_c] = (
            board[next_r][next_c], board[blank_r][blank_c]
        )
        previous_blank = (blank_r, blank_c)
        blank_r, blank_c = next_r, next_c

    if sum(board, []) == tiles:
        board[blank_r][blank_c], board[blank_r - 1][blank_c] = (
            board[blank_r - 1][blank_c], board[blank_r][blank_c]
        )

    return board
```

This creates the solved board and scrambles it using **legal blank moves**, avoiding immediate reversals. Every starting board is therefore reachable, and the final check prevents starting already solved.

2. **Save the file**, then rerun the same testing block you just used. Expected output:

```
3x3: 0/100 boards are unsolvable
4x4: 0/100 boards are unsolvable
5x5: 0/100 boards are unsolvable
```

3. Check gameplay:

```
python main.py
```

Try a few moves, then enter `q` to quit.

4. **If the tests pass**, commit only this task:

```
git add puzzle.py
git diff --cached --stat
git commit -m "Task 1: Guarantee solvable starting boards"
```

Send me the **test results and commit output**, then we'll continue to Task 2.

You said:

i feel it still is the same. can u check i added the new make board func

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> python main.py
Sliding Puzzle — W/A/S/D moves the tile into the blank. Q quits.

   9 14  6
15 10  2  4
 7  8 12  3
13  5  1 11
Moves: 0  Time: 0 s
> d

 9    14  6
15 10  2  4
 7  8 12  3
13  5  1 11
Moves: 1  Time: 4 s
> s

 9 10 14  6
15     2  4
 7  8 12  3
13  5  1 11
Moves: 2  Time: 6 s
> a

 9 10 14  6
  15  2  4
 7  8 12  3
13  5  1 11
Moves: 3  Time: 7 s
> w

  10 14  6
 9 15  2  4
 7  8 12  3
13  5  1 11
Moves: 4  Time: 8 s
> d

10    14  6
 9 15  2  4
 7  8 12  3
13  5  1 11
Moves: 5  Time: 10 s
> d
```

```
10 14     6
 9 15  2  4
 7  8 12  3
13  5  1 11
Moves: 6  Time: 11 s
> q
```

ChatGPT said:

It's expected to look the same—Task 1 changes solvability, not the appearance or controls. The board you pasted **is solvable**, and its moves and move counts look correct.

Previously, tiles were shuffled arbitrarily, which could create an impossible puzzle. Now, the board is scrambled using legal moves from the solved arrangement, so those moves can always be reversed to solve it.

To confirm the fix across many boards, **rerun the solvability testing block** from earlier. All three sizes should report:

```
3x3: 0/100 boards are unsolvable
4x4: 0/100 boards are unsolvable
5x5: 0/100 boards are unsolvable
```

Send me that output before we commit Task 1.

You said:

```
3x3: 0/100 boards are unsolvable
4x4: 0/100 boards are unsolvable
5x5: 0/100 boards are unsolvable
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Task 1 passed: all 300 generated boards were solvable, and you've checked gameplay.

Commit the fix now:

```
git add puzzle.py
git diff --cached --stat
git commit -m "Task 1: Guarantee solvable starting boards"
```

The staged changes should contain **only** `puzzle.py` . Paste the commit output, then we'll begin Task 2.

You said:

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git add puzzle.py
>> git diff --cached --stat
>> git commit -m "Task 1: Guarantee solvable starting boards"
 puzzle.py | 37
++++++++++++++++++++++++++++++++++++--
 1 file changed, 35 insertions(+), 2 deletions(-)
[main 3cb0ce8] Task 1: Guarantee solvable starting boards
 1 file changed, 35 insertions(+), 2 deletions(-)
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Task 1 is committed successfully as `3cb0ce8`.

Now **Task 2: complete the puzzle lifecycle**. Your original code already detects a solved board inside `run()`. We'll add an explicit completion state, freeze the final time, and prevent the completed game from accepting further moves—even if `run()` is called again.

Replace the contents of **`game.py`** with:

```
import time
from puzzle import Puzzle


class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.completed = False
        self.finished = None

    def elapsed(self):
        end = self.finished if self.finished is not None else time.monotonic()
        return end - self.started

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print("Moves:", self.moves, " Time:", int(self.elapsed()), "s")

    def finish_if_solved(self):
```

```
        if not self.completed and self.puzzle.solved():
            self.completed = True
            self.finished = time.monotonic()
        return self.completed

    def run(self):
        if self.completed:
            print("This puzzle is already completed.")
            return

        print("Sliding Puzzle — W/A/S/D moves the blank. Q quits.")
        while True:
            self.finish_if_solved()
            self.display()

            if self.completed:
                print("Solved!")
                return

            key = input("> ").strip().lower()
            if key == "q":
                return
            if key not in "wasd":
                print("Use W/A/S/D.")
                continue
            if self.puzzle.move(key):
                self.moves += 1
            else:
                print("That move is not possible.")
```

The instructions now correctly say the keys move the **blank**, matching the existing movement logic. We'll address invalid-command validation in Task 4.

Save the file, then paste this test into PowerShell. It creates a 3×3 board one move from completion **only in memory**, solves it, and checks that running the completed game again cannot change its state:

```
@'
from unittest.mock import patch
from game import SlidingPuzzle
from puzzle import Puzzle

game = SlidingPuzzle()
game.size = 3
game.puzzle = Puzzle(3)
game.puzzle.board = [
    [1, 2, 3],
    [4, 5, 6],
```

```
    [7, 0, 8],
]

with patch("builtins.input", return_value="d") as mocked_input:
    game.run()
    assert mocked_input.call_count == 1

assert game.puzzle.solved()
assert game.completed
assert game.moves == 1
assert game.finished is not None
print("PASS: Solving the board ends the game after one move.")

board_before = [row[:] for row in game.puzzle.board]
moves_before = game.moves
elapsed_before = game.elapsed()

with patch("builtins.input", side_effect=AssertionError("Unexpected input")):
    game.run()

assert game.puzzle.board == board_before
assert game.moves == moves_before

with patch("game.time.monotonic", return_value=game.finished + 100):
    assert game.elapsed() == elapsed_before

print("PASS: Completed game accepts no further commands.")
print("PASS: Board and move count remain unchanged.")
print("PASS: Final elapsed time stays frozen.")
'@ | python -
```

Send me the test output before committing.

You said:

```
>> '@ | python -
Sliding Puzzle — W/A/S/D moves the blank. Q quits.

 1  2  3
 4  5  6
 7     8
Moves: 0  Time: 0 s

 1  2  3
 4  5  6
 7  8
```

```
Moves: 1  Time: 0 s
Solved!
PASS: Solving the board ends the game after one move.
This puzzle is already completed.
PASS: Completed game accepts no further commands.
PASS: Board and move count remain unchanged.
PASS: Final elapsed time stays frozen.
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Task 2 passed: the game ends when solved, accepts no further commands, and freezes the final time.

Commit it now:

```
git add game.py
git diff --cached --stat
git commit -m "Task 2: Complete puzzle lifecycle and freeze final time"
```

The staged changes should contain **only** `game.py`. Paste the commit output, then we'll add the size modes and session tracking in Task 3.

You said:

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git add game.py
>> git diff --cached --stat
>> git commit -m "Task 2: Complete puzzle lifecycle and freeze final
time"
 game.py | 27 ++++++++++++++++++++++++----
 1 file changed, 23 insertions(+), 4 deletions(-)
[main 0afae03] Task 2: Complete puzzle lifecycle and freeze final
time
 1 file changed, 23 insertions(+), 4 deletions(-)
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Task 2 is committed as `0afae03`.

For **Task 3**, we'll add:

- A choice of **3×3, 4×4, or 5×5** before playing.
- Consistent move counts and timing across all sizes.
- An **`r` command to recreate the board while keeping the current session's moves and elapsed time**. Starting the program again starts a fresh session.

1. Replace `game.py` with:

```
import time
from puzzle import Puzzle


class SlidingPuzzle:
    def __init__(self, size=4):
        if size not in (3, 4, 5):
            raise ValueError("Board size must be 3, 4, or 5.")

        self.size = size
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.completed = False
        self.finished = None

    def recreate_board(self):
        if self.completed:
            return False
        self.puzzle = Puzzle(self.size)
        return True

    def elapsed(self):
        end = self.finished if self.finished is not None else time.monotonic()
        return end - self.started

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print("Moves:", self.moves, " Time:", int(self.elapsed()), "s")

    def finish_if_solved(self):
        if not self.completed and self.puzzle.solved():
            self.completed = True
            self.finished = time.monotonic()
        return self.completed

    def run(self):
        if self.completed:
            print("This puzzle is already completed.")
            return

        print(f"Sliding Puzzle – {self.size}x{self.size}")
        print("W/A/S/D moves the blank. Q quits.")
```

```
        print("R recreates the board; session moves and time continue.")

        while True:
            self.finish_if_solved()
            self.display()

            if self.completed:
                print("Solved!")
                return

            key = input("> ").strip().lower()
            if key == "q":
                return
            if key == "r":
                self.recreate_board()
                print("Board recreated. Session moves and time retained.")
                continue
            if key not in "wasd":
                print("Use W/A/S/D, R, or Q.")
                continue
            if self.puzzle.move(key):
                self.moves += 1
            else:
                print("That move is not possible.")
```

2. Replace `main.py` with:

```
from game import SlidingPuzzle


def main():
    while True:
        choice = input(
            "Choose board size: 3, 4, or 5 (Enter for 4; Q quits): "
        ).strip().lower()

        if choice == "q":
            return
        if choice == "":
            choice = "4"
        if choice in ("3", "4", "5"):
            SlidingPuzzle(int(choice)).run()
            return

        print("Please enter 3, 4, or 5.")
```

```
if __name__ == "__main__":
    main()
```

3. Save both files, then run this test block:

```
@'
from unittest.mock import patch
from game import SlidingPuzzle
from main import main

for size in (3, 4, 5):
    with patch("game.time.monotonic", return_value=100.0):
        game = SlidingPuzzle(size)

    assert len(game.puzzle.board) == size
    assert all(len(row) == size for row in game.puzzle.board)
    assert game.moves == 0

    game.moves = 7
    old_puzzle = game.puzzle

    with patch("game.time.monotonic", return_value=112.0):
        assert game.elapsed() == 12.0
        assert game.recreate_board()
        assert game.puzzle is not old_puzzle
        assert game.moves == 7
        assert game.started == 100.0
        assert game.elapsed() == 12.0

    tiles = list(range(1, size * size)) + [0]
    game.puzzle.board = [
        tiles[r * size:(r + 1) * size]
        for r in range(size)
    ]
    game.puzzle.board[-1][-2:] = [0, size * size - 1]

    with patch("builtins.input", return_value="d"):
        with patch("game.time.monotonic", return_value=120.0):
            game.run()

    assert game.completed
    assert game.moves == 8
    assert game.elapsed() == 20.0
    assert not game.recreate_board()
    assert game.puzzle.solved()
```

```
    print(f"PASS: {size}x{size} size, moves, timer, and recreation.")

for choice in ("3", "4", "5", ""):
    with patch("builtins.input", return_value=choice):
        with patch("main.SlidingPuzzle") as constructor:
            main()
            constructor.assert_called_once_with(int(choice or "4"))
            constructor.return_value.run.assert_called_once()

with patch("builtins.input", side_effect=["2", "hello", "q"]):
    with patch("main.SlidingPuzzle") as constructor:
        main()
        constructor.assert_not_called()

print("PASS: Size selection, default size, invalid sizes, and quit.")
'@ | python -
```

4. Check it manually:

```
python main.py
```

Choose `3`, make a valid move, wait a few seconds, then enter `r`. The board should change, **the move count should stay the same**, and time should continue. Quit with `q`.

Send me the test output and whether that manual check worked; then we'll commit Task 3.

You said:

```
Sliding Puzzle — 3x3
W/A/S/D moves the blank. Q quits.
R recreates the board; session moves and time continue.

 1  2  3
 4  5  6
 7     8
Moves: 7  Time: 20 s

 1  2  3
 4  5  6
 7  8
Moves: 8  Time: 20 s
Solved!
PASS: 3x3 size, moves, timer, and recreation.
Sliding Puzzle — 4x4
W/A/S/D moves the blank. Q quits.
R recreates the board; session moves and time continue.
```

```
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14    15
Moves: 7  Time: 20 s

 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14 15
Moves: 8  Time: 20 s
Solved!
PASS: 4x4 size, moves, timer, and recreation.
Sliding Puzzle — 5x5
W/A/S/D moves the blank. Q quits.
R recreates the board; session moves and time continue.

 1  2  3  4  5
 6  7  8  9 10
11 12 13 14 15
16 17 18 19 20
21 22 23    24
Moves: 7  Time: 20 s

 1  2  3  4  5
 6  7  8  9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24
Moves: 8  Time: 20 s
Solved!
PASS: 5x5 size, moves, timer, and recreation.
Please enter 3, 4, or 5.
Please enter 3, 4, or 5.
PASS: Size selection, default size, invalid sizes, and quit.
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>

(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> python main.py
Choose board size: 3, 4, or 5 (Enter for 4; Q quits): 3
Sliding Puzzle — 3x3
W/A/S/D moves the blank. Q quits.
R recreates the board; session moves and time continue.

 6  7  1
 3  8  2
```

```
   4  5
Moves: 0  Time: 0 s
> d

 6  7  1
 3  8  2
 4     5
Moves: 1  Time: 25 s
> r
Board recreated. Session moves and time retained.

 8  7  4
 1     6
 5  3  2
Moves: 1  Time: 34 s
> q
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Task 3 passed. All sizes passed the automated checks, and your manual test confirms that recreating the board preserves **1 move** while time continues from **25 to 34 seconds**.

Commit both files:

```
git add game.py main.py
git diff --cached --stat
git commit -m "Task 3: Add board sizes and preserve session moves and timer"
```

The staged changes should contain **only `game.py` and `main.py`** . Paste the commit output, then we'll complete Task 4: valid-action feedback and invalid-command handling.

You said:

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git add game.py
main.py
>> git diff --cached --stat
>> git commit -m "Task 3: Add board sizes and preserve session
moves and timer"
 game.py | 24 ++++++++++++++++++++----
 main.py | 20 ++++++++++++++++++-
 2 files changed, 39 insertions(+), 5 deletions(-)
[main f5a89ab] Task 3: Add board sizes and preserve session moves
and timer
 2 files changed, 39 insertions(+), 5 deletions(-)
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

ChatGPT said:

Task 3 is committed as `f5a89ab`.

Now **Task 4: accurate move feedback and invalid-command handling**. The current check `key not in "wasd"` accidentally accepts empty input and substrings such as `wa`, which can crash the game.

1. In `game.py`, replace this section:

```
            if key not in "wasd":
                print("Use W/A/S/D, R, or Q.")
                continue
            if self.puzzle.move(key):
                self.moves += 1
            else:
                print("That move is not possible.")
```

With:

```
            if key not in ("w", "a", "s", "d"):
                print("Use W/A/S/D, R, or Q.")
                continue
            if self.puzzle.move(key):
                self.moves += 1
                print("Slide successful.")
            else:
                print("That move is not possible.")
```

2. In `puzzle.py`, add validation at the beginning of `move()`. The complete method should now be:

```
    def move(self, direction):
        directions = {
            "w": (-1, 0),
            "s": (1, 0),
            "a": (0, -1),
            "d": (0, 1),
        }

        if direction not in directions:
            return False

        r, c = self.blank_pos()
        dr, dc = directions[direction]
        nr, nc = r + dr, c + dc

        if not (0 <= nr < self.size and 0 <= nc < self.size):
            return False
```

```
        self.board[r][c], self.board[nr][nc] = (
            self.board[nr][nc], self.board[r][c]
        )
        return True
```

3. Save both files and run this test block:

```
@'
import io
from contextlib import redirect_stdout
from unittest.mock import patch
from game import SlidingPuzzle
from puzzle import Puzzle

for size in (3, 4, 5):
    puzzle = Puzzle(size)
    tiles = list(range(size * size))
    puzzle.board = [
        tiles[r * size:(r + 1) * size]
        for r in range(size)
    ]
    original = [row[:] for row in puzzle.board]

    for command in ("w", "a", "", "wa", "wasd", "x"):
        assert puzzle.move(command) is False
        assert puzzle.board == original

    assert puzzle.move("d") is True
    assert puzzle.blank_pos() == (0, 1)
    assert puzzle.move("a") is True
    assert puzzle.board == original
    print(f"PASS: {size}x{size} legal, boundary, and invalid moves.")

game = SlidingPuzzle(3)
game.puzzle.board = [
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8],
]

commands = ["w", "a", "", "wa", "wasd", "x", " D ", "q"]
output = io.StringIO()

with redirect_stdout(output):
    with patch("builtins.input", side_effect=commands):
```

```
        game.run()

text = output.getvalue()
assert game.moves == 1
assert game.puzzle.board == [
    [1, 0, 2],
    [3, 4, 5],
    [6, 7, 8],
]
assert text.count("Slide successful.") == 1
assert text.count("That move is not possible.") == 2
assert text.count("Use W/A/S/D, R, or Q.") == 4

print("PASS: Only successful slides increment moves and report success.")
print("PASS: Empty and multi-letter commands are rejected safely.")
print("PASS: Uppercase input, surrounding spaces, and quit work.")
'@ | python -
```

Send me the output before we commit Task 4.

You said:

PASS: 3x3 legal, boundary, and invalid moves.
PASS: 4x4 legal, boundary, and invalid moves.
PASS: 5x5 legal, boundary, and invalid moves.
PASS: Only successful slides increment moves and report success.
PASS: Empty and multi-letter commands are rejected safely.
PASS: Uppercase input, surrounding spaces, and quit work.
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>

ChatGPT said:

Task 4 passed: valid slides give success feedback and increment moves; impossible moves and invalid commands leave the board and count unchanged.

Commit it now:

```
git add game.py puzzle.py
git diff --cached --stat
git commit -m "Task 4: Validate commands and report successful slides accurately"
```

The staged changes should contain **only `game.py` and `puzzle.py`** .

Paste the commit output. Then we'll check the final submission requirements, record the **10-second "after" video**, and prepare the files and chat history for upload.

You said:

(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git add game.py puzzle.py
>> git diff --cached --stat
>> git commit -m "Task 4: Validate commands and report successful slides accurately"
 game.py | 3 ++-
 puzzle.py | 18 ++++++++++++++++++--
 2 files changed, 18 insertions(+), 3 deletions(-)
[main 888627c] Task 4: Validate commands and report successful slides accurately
 2 files changed, 18 insertions(+), 3 deletions(-)
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>

ChatGPT said:

All four tasks are now tested and committed. Next, record the **10-second "after" video**.

1. Run:

   ```
   python main.py
   ```

2. Choose `3`, then start recording with **Windows + Shift + R**, selecting the terminal area.
3. During the recording:
   - Make a valid move to show "**Slide successful.**" and the increased move count.
   - Enter `x` to show invalid-command feedback without increasing moves.
   - Enter `r` to show the recreated board retaining moves and elapsed time.
   - Enter `q` to quit.
4. Stop recording at **10 seconds** and save it as **`after.mp4`** in your project folder.

Your earlier `before.mp4` was **16.7 seconds**—trim it to a 10-second gameplay segment as well.

Once both videos are ready, run:

```
git status
git log -6 --oneline
```

Paste the output. Then we'll push your commits and prepare the **`Lab-4` submission folder**, including both videos, updated code, complete chat link, and exported chat history.

You said:

i closed all the vs code terminals and vs code itaelf. give the command for venv

ChatGPT said:

Open the project folder in VS Code, open a new PowerShell terminal, then run:

```
.\venv\Scripts\Activate.ps1
```

You should see `(venv)` at the beginning of the terminal prompt.

You said:

after recording after.mp4
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git status
>> git log -6 --oneline
On branch main
Your branch is ahead of 'origin/main' by 4 commits.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/
        after.mp4
        before.mp4
        venv/

nothing added to commit but untracked files present (use "git add" to track)
888627c (HEAD -> main) Task 4: Validate commands and report successful slides accurately
f5a89ab Task 3: Add board sizes and preserve session moves and timer
0afae03 Task 2: Complete puzzle lifecycle and freeze final time
3cb0ce8 Task 1: Guarantee solvable starting boards
75858c1 (origin/main, origin/HEAD) Updated scenario title
56c88a9 Initial commit
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>

Worked for 9s

ChatGPT said:

Everything is correct: all four task commits are ready, and the videos are currently untracked.

Push the updated code to your fork:

```
git push origin main
```

After it finishes, run:

`git status`

Do **not** run `git add .` ; that would include `venv/` and `__pycache__/` .

Paste the push and status output here. After that, we'll verify both video durations and prepare the exact `Lab-4` submission folder.

You said:

```
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git push origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 2.87 KiB | 195.00 KiB/s, done.
Total 14 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 1 local object.
To https://github.com/vishalh06/22_sliding_puzzle.git
   75858c1..888627c  main -> main
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/
        after.mp4
        before.mp4
        venv/

nothing added to commit but untracked files present (use "git add" to track)
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

Worked for 13s

ChatGPT said:

The code is successfully pushed to your fork, and all four task commits are visible online.

Now check both video durations:

1. Open the project folder in File Explorer.
2. Right-click each video → **Properties** → **Details** → **Length**.
3. Both should be approximately **10 seconds**. Trim `before.mp4` , since the version checked earlier was 16.7 seconds.

Then attach the final `before.mp4` and `after.mp4` here so I can verify their duration and visible content before we prepare the submission folder.

You said:

i just made the trim in file explorer, i want to push

Worked for 8s

ChatGPT said:

Don't push the videos to `22_sliding_puzzle` yet. Your instructions say the deliverables must be uploaded to the **`Lab-4` folder of your Lab repository**, while this repository is the scenario fork containing your updated code.

Send me the **GitHub link or local folder path of your Lab repository**, and I'll give you the exact commands to add:

```
Lab-4/
├── before.mp4
├── after.mp4
├── updated code
└── chat history PDF/DOCX
```

You said:

https://github.com/vishalh06/22_sliding_puzzle

Worked for 16s

ChatGPT said:

Confirmed—this is the repository where the README requires the two gameplay videos and chat-history link.

Push the trimmed videos with:

```
git add before.mp4 after.mp4
git diff --cached --stat
git commit -m "Add before and after gameplay videos"
git push origin main
```

This stages only the two videos, so `venv/` and `__pycache__/` will remain untracked. Paste the output afterward; then we'll add your complete ChatGPT conversation link.

You said:

```
ok done lets add history
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> git add before.mp4
after.mp4
>> git diff --cached --stat
>> git commit -m "Add before and after gameplay videos"
>> git push origin main
 after.mp4  | Bin 0 -> 9901234 bytes
 before.mp4 | Bin 0 -> 1861219 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
[main 76b81f5] Add before and after gameplay videos
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 after.mp4
 create mode 100644 before.mp4
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.23 MiB | 1.41 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/vishalh06/22_sliding_puzzle.git
   888627c..76b81f5  main -> main
(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle>
```

Worked for 9s

ChatGPT said:

The videos are pushed successfully. Now add both the **complete chat link** required by the README and the **PDF export** required by your lab instructions.

1. In this ChatGPT conversation, click **Share** at the top-right → **Create link** → **Copy link**.
2. In PowerShell, run:

```
$link = Read-Host "Paste the complete ChatGPT conversation link"
```

Paste the link and press Enter. Then run:

```
Set-Content -Path "chat_history_link.txt" -Value $link
```

3. Export this conversation by pressing **Ctrl + P** in your browser:
   - Destination: **Save as PDF**
   - Filename: `Chat_History.pdf`
   - Save it inside `D:\pes\SE_LAB4\22_sliding_puzzle`

4. Commit and push both:

```
git add chat_history_link.txt Chat_History.pdf
git diff --cached --stat
git commit -m "Add complete LLM chat history"
git push origin main
```

Paste the output afterward, and we'll perform the final repository check.

You said:

im not able to paste

(venv) PS D:\pes\SE_LAB4\22_sliding_puzzle> $link = Read-Host "Paste the complete ChatGPT conversation link"
Paste the complete ChatGPT conversation link: ^V

ChatGPT said:

`Ctrl + V` may not paste into your PowerShell terminal. First press **Ctrl + C** to cancel the current `Read-Host` command.

Then copy the ChatGPT share link again and paste it using one of these:

- **Right-click** inside the terminal.
- Press **Ctrl + Shift + V**.
- Press **Shift + Insert**.

Run again:

```
$link = Read-Host "Paste the complete ChatGPT conversation link"
```

After pasting, press **Enter**, then run:

```
Set-Content -Path "chat_history_link.txt" -Value $link
Get-Content "chat_history_link.txt"
```

The final command should display your complete ChatGPT share link.